TESTIFY.ID

Business Requirements and Technical Documentation

Current-state product baseline

Purpose A concise, implementation-grounded reference for product scope, business behavior, data design, APIs, security, deployment, and quality assurance.

Document status	Baseline	Prepared
Approved draft for review	Commit cd2677a on main	06 July 2026

Prepared from the current Testify repository. Runtime routes and checked-in migrations are treated as authoritative where documentation differs.

01 / ORIENTATION

Document Control and Executive Summary

Item	Definition
Product	Testify, a project-scoped quality assurance workspace.
Primary audience	Product owner, QA lead, software engineer, API integrator, operations team, and deployment reviewer.
System boundary	Authenticated web application, versioned API, relational database, object/local file storage, and optional AI provider.
Source of truth	Current application code, migrations, models, services, policies, routes, tests, and runtime configuration.
Change control	Update this document when business states, relationships, API contracts, storage strategy, or ownership rules change.

Executive summary

Testify connects the core QA lifecycle in one workspace: organize a project, maintain reusable test cases, assemble plans, execute selected cases, capture defects, and communicate release readiness. The current tenancy model is deliberately simple: each project has one owner, and every business object is scoped to that owner's active project.

Business value

- Creates traceability from test intent through execution and defects.
- Reduces fragmented spreadsheets and duplicated case maintenance.
- Provides consistent priority, severity, status, and environment taxonomies.
- Supports stakeholder reporting through dashboard and PDF exports.

Technical posture

- Laravel domain services and policies enforce ownership and transactions.
- Inertia and Svelte provide a server-driven SPA experience.
- A strict JSON:API v1 supports token-based integrations.
- Soft deletes, optimistic concurrency, idempotency, and audit logs protect API writes.

Current-state boundary Browser automation generation and execution are not part of the current application. Test Case automation_status is classification metadata only. Automated browser scripts are expected to remain external until a separate integration is designed.

02 / PRODUCT SCOPE

Business Requirements Document

Problem statement

QA work is often split across documents, issue trackers, and isolated execution records. Teams lose traceability, repeat maintenance, and struggle to explain release risk.

Product vision

Provide a focused workspace in which a QA owner can manage the complete test lifecycle for each project and produce evidence-based release reporting.

Primary stakeholders

- Project owner / QA lead - owns project data and release decisions.
- Tester - authors cases and records execution outcomes.
- Developer / triage partner - consumes issue evidence and expected behavior.
- Stakeholder / manager - reviews dashboard and exported reports.
- API integrator - exchanges QA data through scoped tokens.

Business objectives and measures

Objective	Required outcome	Indicative measure
Centralize QA assets	Projects, cases, plans, runs, issues, and reports share one data model.	Reduced duplicate test records and manual reconciliation.
Improve traceability	A plan selects repository cases; a run executes plan cases; issues can reference the plan and run.	Every release report can trace risk to execution and defects.
Protect project isolation	A user cannot access resources owned by another user or outside the active project.	Cross-user and cross-project requests return no data / 404.
Accelerate authoring	AI may produce reviewable drafts for plans, cases, issues, and report summaries.	Less authoring time without bypassing human review.
Support integration	Production API operations are versioned, rate-limited, auditable, and concurrency-safe.	Stable automated exchange with replay and stale-write protection.

Scope

In scope	Out of scope / current limitation
Authentication, profile, password/security settings; single-owner projects; active project workspace; repository modules and cases; test plans; manual test execution; issues and image evidence; dashboard; report/PDF export; AI drafting; JSON:API v1; durable data migration tooling.	Project memberships and role-based team administration; live collaboration; notifications; public sharing; restore UI; bulk API operations; automatic issue creation; internal browser automation; arbitrary Playwright scripts; multi-file issue attachments; production data warehouse.

03 / CAPABILITIES

Functional Requirements and Business Rules

ID	Area	Requirement
FR-01	Identity	Users authenticate, manage their profile and credentials, and may use expiring Sanctum API tokens.
FR-02	Projects	An authenticated user creates, updates, selects, archives, and deletes only owned projects.
FR-03	Repository	The owner organizes hierarchical modules and creates reusable test cases with ordered action/expected-result steps.
FR-04	Case keys	The system generates unique project case keys from the module prefix inside a locked transaction.
FR-05	Test plans	The owner defines release scope, strategy, criteria, schedule, priority, tester, and selected repository cases.
FR-06	Test runs	A run selects a plan and a non-empty subset of cases already assigned to that plan.
FR-07	Execution	A run must be started before case results change. Result updates maintain the run lifecycle automatically.
FR-08	Issues	The owner records defect context, impact, expected/actual result, assignment metadata, and optional plan/run links.
FR-09	Evidence	An issue may contain one JPG, PNG, or WebP image up to 5 MB, stored on the configured filesystem disk.
FR-10	Analytics	Dashboard and reports aggregate only the active project's plans, runs, case outcomes, workload, and issues.
FR-11	AI assistance	AI returns normalized, reviewable drafts; invalid model values are replaced with safe domain defaults.
FR-12	API	Owned resources support versioned CRUD, relationships, run start/result actions, pagination, filters, includes, and sparse fields.

Cross-cutting business rules

Rule	Enforcement
Ownership	Policies and owned queries restrict projects and descendants to the authenticated user.
Active project	The web session key qa_workspace_project_id scopes repository, plans, executions, issues, dashboard, and reports.
Referential consistency	Modules, cases, plans, runs, and issue links must resolve to the same project.
Transactional integrity	Case plus steps, run plus selected cases, and issue plus attachment changes are atomic.
Deletion	Primary QA resources use soft deletion; relationship rows and steps follow database cascade/restrict rules.
Human accountability	Tester, reporter, and assignee are currently free-text fields, not user foreign keys.

04 / USER NEEDS

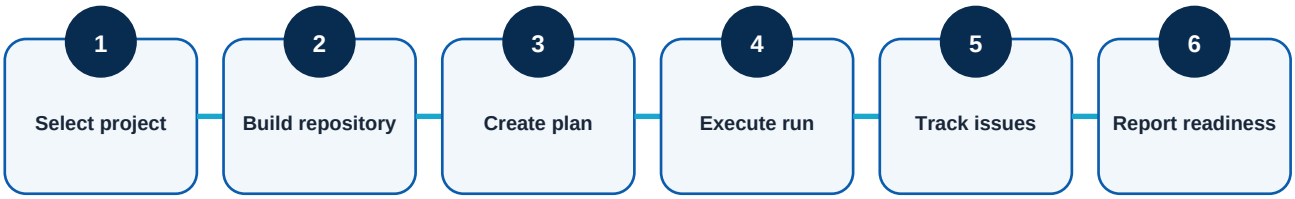
User Stories and Acceptance Outcomes

ID	Actor	Story	Acceptance outcome
US-01	Project owner	Select an owned project workspace	All downstream lists and actions show only that project; deleting it clears the active context.
US-02	QA lead	Organize cases in nested repository modules	Parent-child modules remain inside one project; a parent with children cannot be silently removed.
US-03	Tester	Create a case with ordered steps	Case key is generated, steps persist in order, and partial failure rolls back the case.
US-04	QA lead	Reuse cases across plans	Only same-project cases can be attached; duplicate attachment is prevented.
US-05	QA lead	Define a release test plan	Dates are coherent, states and priorities are allowlisted, and PDF export is available.
US-06	Tester	Create and start a test run	The run contains at least one case from its plan and captures start time when execution begins.
US-07	Tester	Record each case outcome	Only attached cases can be changed; the run completes when every selected case is Passed.
US-08	Tester	Report a defect with evidence	Issue links remain in the active project and the optional image passes type and size controls.
US-09	QA lead	Review release health	Dashboard and reports show project-scoped coverage, results, issue risk, workload, and recent runs.
US-10	QA author	Request an AI draft	The draft uses saved project context, remains editable, and never writes business data without user submission.
US-11	API integrator	Create a case and its steps in one request	The operation is atomic, idempotent, auditable, and returns a JSON:API resource document.
US-12	Security owner	Prevent data leakage and stale writes	Cross-owner resources are hidden, abilities are enforced, and If-Match rejects stale mutations.

Acceptance principle A feature is complete only when ownership, active-project scoping, validation, transaction behavior, and regression coverage agree across the web app and API.

05 / OPERATING MODEL

Business Flow and Feature Relationships



Project context and ownership are enforced at every step

Feedback loop: failed execution -> issue -> fix -> re-run -> updated report

End-to-end flow

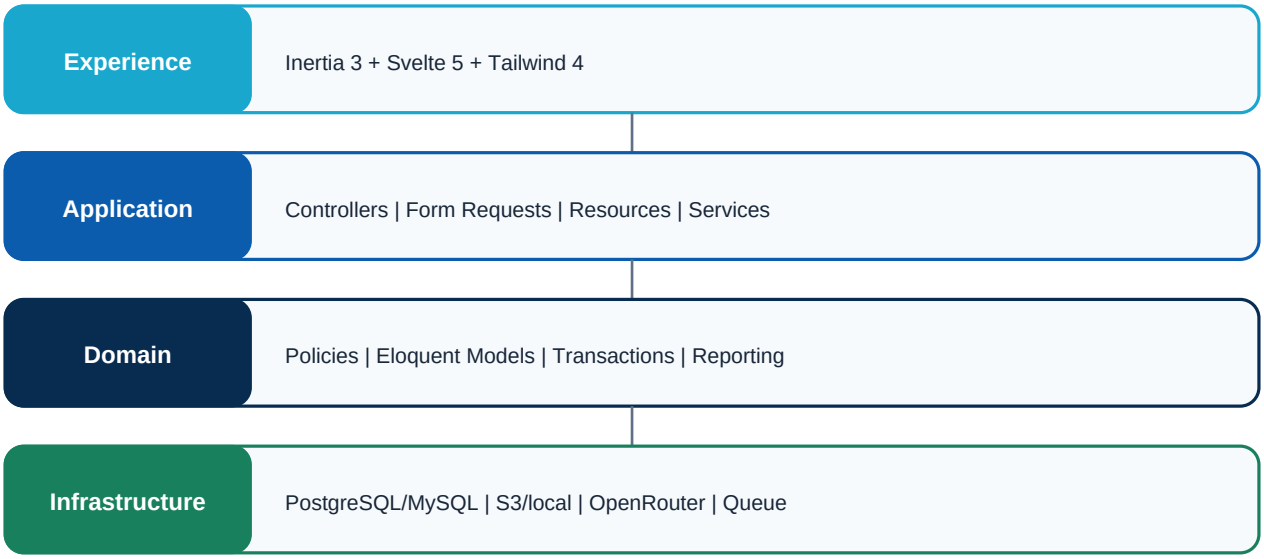
Stage	User action	System behavior / output
1. Establish context	Sign in, create or choose a project.	Stores the active project in session and scopes every QA screen.
2. Design coverage	Create modules, cases, preconditions, and steps.	Generates stable case keys and maintains reusable repository assets.
3. Plan a release	Create a plan and attach relevant cases.	Defines what is in scope, when it is due, and who is responsible.
4. Execute	Create a run, start it, and update each case result.	Captures environment, tester, timestamps, and the latest outcome per selected case.
5. Triage defects	Create an issue, optionally linking its plan/run and image.	Validates project lineage, stores evidence, and tracks severity/status.
6. Decide readiness	Review dashboard or filtered report; export PDF.	Combines coverage, execution distribution, issue trend, workload, and recent activity.
7. Iterate	Fix, re-test, and update outcomes/issues.	Preserves the connected history and refreshes readiness metrics.

Feature dependency map

Feature	Depends on	Feeds
Projects	Authenticated user	All QA modules and active workspace
Repository	Project and module hierarchy	Plans, runs, AI context
Test plans	Project and repository cases	Runs, issues, plan PDF, reports
Test executions	Plan and assigned plan cases	Case results, issues, dashboards, reports
Issues	Project; optional plan/run	Risk metrics, trends, release reporting
AI drafts	User prompt plus authorized project context	Editable plan/case/issue/report content

06 / SYSTEM DESIGN

Technical Architecture



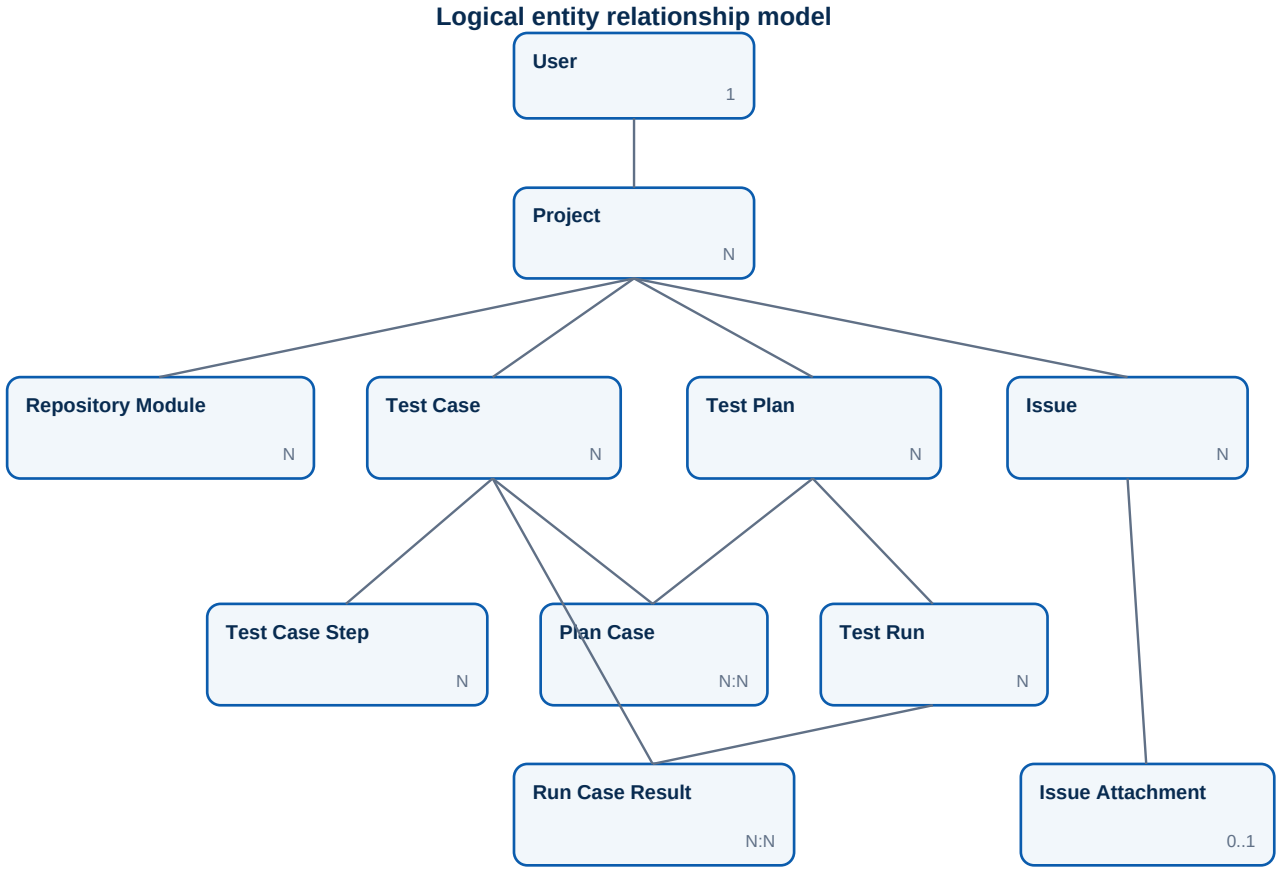
Layer	Current implementation	Responsibility
Web experience	Inertia.js 3, Svelte 5, Tailwind CSS 4, Wayfinder	Authenticated pages, forms, modal workflows, typed route calls, responsive presentation.
HTTP/application	Laravel 13 controllers, Form Requests, resources, middleware	Routing, input validation, content negotiation, authentication, response shaping.
Domain	Eloquent models, policies, focused services	Ownership, state transitions, aggregate queries, relationship integrity, transactions.
API foundation	Sanctum, JSON:API middleware, audit/idempotency services	Bearer abilities, request IDs, ETags, replay safety, consistent errors.
Persistence	MySQL/PostgreSQL-compatible migrations and durable import/export commands	Relational integrity, soft deletes, API versions, sequence reset after import.
Files	Laravel local or S3-compatible disk	Private issue evidence and generated PDF delivery.
AI	Laravel AI with OpenRouter configuration	Structured QA draft generation with normalization and event metadata logging.

Primary code boundaries

Pages live under resources/js/pages. Web and session API controllers live under app/Http/Controllers. Production API v1 controllers, middleware, JSON:API support, audit, and idempotency components are isolated under Api/V1 namespaces. Business logic is concentrated in app/Services, while ownership decisions reside in app/Policies.

07 / DATA ARCHITECTURE

Database Relationship Model



Relationship rules

Relationship	Cardinality	Delete behavior / constraint
User -> Project	1:N	Project rows are owned by user_id; database cascade on hard delete.
Project -> Module / Plan / Case / Issue	1:N	All are project-scoped; principal resources also support soft delete.
Module -> Module	1:N recursive	parent_id supports hierarchy; parent deletion is restricted.
Module -> Test Case	1:N	A case must reference a module in the same project; module hard deletion is restricted.
Test Case -> Step	1:N ordered	position is unique per case; hard deletion cascades.
Test Plan <-> Test Case	N:N	Unique plan/case pair in test_case_test_plan.
Test Plan -> Test Run	1:N	A run inherits project scope through its plan; hard deletion cascades.
Test Run <-> Test Case	N:N with result	Pivot stores execution_status and unique run/case pair.
Issue -> Plan / Run	N:0..1	Links are optional and become null on referenced hard deletion.
Issue -> Attachment	1:0..1	Unique issue_id guarantees one image; hard deletion cascades.

08 / PHYSICAL SCHEMA

Database Schema - Core Planning

Table	Key columns	Purpose / constraints
users	id PK; name; email UQ; email_verified_at; password; remember_token; is_active; timestamps	Identity and project owner.
projects	project_id PK; user_id FK; project_name; project_description; project_url; project_status; api_version; deleted_at; timestamps	Top-level tenant boundary.
test_repository_modules	id PK; project_id FK; parent_id FK nullable; name; key_prefix; sort_order; api_version; deleted_at; timestamps	Nested repository folders and case-key prefix.
test_cases	id PK; project_id FK; module_id FK; case_key; title; priority; automation_status; test_behavior; test_type; preconditions; api_version; deleted_at; timestamps	Reusable case metadata. UQ(project_id, case_key).
test_case_steps	id PK; test_case_id FK; position; action; expected_result; timestamps	Ordered procedure. UQ(test_case_id, position).
test_plans	id PK; project_id FK; name; description; scope; strategy; criteria; status; priority; start_date; due_date; release_version; tester; api_version; deleted_at; timestamps	Release/sprint test intent.
test_case_test_plan	id PK; test_plan_id FK; test_case_id FK; timestamps	Plan coverage membership. UQ(test_plan_id, test_case_id).

Authoritative enum values

Domain	Allowed values
Project status	Active, In Progress, Archived
Plan status	Draft, In Progress, Completed, Archived
Priority	Low, Medium, High, Urgent
Case automation classification	Manual, Automated (classification only; no internal runner)
Case behavior	Positive, Negative
Case type	Acceptance, Accessibility, Compatibility, Functional, Integration, Performance, Regression, UI/UX, Security, Sanity, Smoke

Concurrency Case key generation queries the latest key for a module prefix inside a database transaction with row locking. This prevents duplicate keys during concurrent creation.

09 / PHYSICAL SCHEMA

Database Schema - Execution and Defects

Table	Key columns	Purpose / constraints
test_runs	id PK; test_plan_id FK; name; environment; priority; description; tester_name; status; execution_started_at; execution_completed_at; api_version; deleted_at; timestamps	Execution aggregate derived from one plan.
test_case_test_run	id PK; test_run_id FK; test_case_id FK; execution_status; timestamps	Per-case execution result. UQ(test_run_id, test_case_id).
issues	id PK; project_id FK; test_plan_id FK nullable; test_run_id FK nullable; title; description; expected_result; actual_result; priority; severity; status; reporter; assignee; environment; defect_type; closed_at; api_version; deleted_at; timestamps	Defect and triage record with optional traceability.
issue_attachments	id PK; issue_id FK UQ; disk; path; original_name; mime_type; size; timestamps	One private image evidence file per issue.

Execution state model

Object	States	Transition rule
Test run	New -> In Progress -> Completed	Start changes New to In Progress and records start time. Completed runs cannot be started. All selected cases Passed completes the run; a later non-passed result returns it to In Progress.
Case result	Not Run, In Testing, Passed, Failed, Skipped, Blocked	Result can change only after the run starts and only for a case assigned to that run.
Issue	Open, In Investigation, In Testing, Solved, Deferred	Solved records closed_at; moving away from Solved clears closed_at.

Classification values

Domain	Allowed values
Environment	Development, SIT, Staging, UAT, Production
Severity	Low, Medium, High, Critical
Defect type	UI/UX, Functional, Compatibility, Database, Integration, Performance, Security, Usability
Issue evidence	JPG/JPEG, PNG, or WebP; maximum 5 MB; one file per issue

10 / SUPPORTING DATA

Database Schema - Platform and Operations

Table(s)	Contents	Retention / role
personal_access_tokens	Sanctum token owner, hash, abilities, usage and expiry	Durable; migrated to preserve API continuity.
api_idempotency_keys	User/token, key, request hash, stored response, expiry	POST replay protection; response retained for the configured window.
api_audit_logs	User/token, UUID request_id, action, resource, before/after, client metadata	Immutable API activity evidence; secrets are redacted.
agent_conversations	UUID id, optional user, title, timestamps	Laravel AI conversation header.
agent_conversation_messages	UUID id, conversation, user, agent, role, content, tool/usage/meta payloads	Laravel AI conversation detail.
sessions	Session id, user, client metadata, payload, last activity	Operational; recreated at migration/cutover.
cache / cache_locks	Key/value/expiry and lock owner	Ephemeral operational state.
jobs / job_batches / failed_jobs	Queue payload, attempts, timing, failures	Ephemeral queue state; failed jobs retained for diagnosis.
password_reset_tokens	Email, token, timestamp	Short-lived identity recovery state.
migrations	Applied migration ledger	Environment-specific; never copied as business data.

Durable migration scope

The built-in export/import workflow preserves users, all QA business tables, attachments metadata, Sanctum tokens, API safety/audit tables, and AI conversation tables. Sessions, cache, queues, password resets, and the migration ledger are intentionally recreated. PostgreSQL sequences are reset after import to avoid key collisions.

Data integrity and retention

Control	Implementation
Soft deletion	projects, repository modules, plans, cases, runs, and issues retain deleted_at and are hidden by default.
Optimistic concurrency	api_version increments on API-visible mutations and supports ETag / If-Match checks.
Foreign keys	Project lineage uses constrained keys; pivots enforce unique memberships; module and case constraints prevent orphaning.
Attachment durability	Database stores disk/key metadata; bytes live on local private storage or an S3-compatible provider.

11 / INTEGRATION

Interfaces and API Contract

Web/session interface

- Authenticated Inertia pages use Laravel sessions and CSRF protection.
- Legacy /api endpoints support the current SPA and remain project-scoped.
- A 419 response uses code session_expired and a login recovery URL.
- PDF exports are authenticated and rate-limited.

Production API v1

- Base path /api/v1 and media type application/vnd.api+json.
- Sanctum Bearer tokens expire after 30 days by default.
- String resource IDs, ISO-8601 timestamps, JSON:API errors.
- Cursor pagination, allowlisted filters/sort/include/sparse fields.
- Request IDs, ETags, If-Match, idempotency, abilities, and audit logs.

Current API resource groups

Group	Operations
Authentication and profile	Create token; revoke current token; show/update current user; list/revoke owned tokens.
Projects	List, detail, create, update, soft-delete.
Repository modules	List, detail, create, update, soft-delete.
Test plans	CRUD plus list/attach/detach test-case relationships.
Test cases	CRUD; create/update can atomically include the complete ordered step list.
Test runs	CRUD, start action, and per-case result update.
Issues	CRUD with owned project/plan/run validation.
Documentation	Authenticated OpenAPI download when API_DOCS_ENABLED is true.

Rate limits and token abilities

Defaults are 5 login attempts/minute, 120 reads/minute, and 60 writes/minute. Abilities are separated by resource read/write scopes, plus profile, token, and documentation scopes. Abilities never bypass ownership policies.

Documentation gap docs/openapi/v1.yaml still references removed browser-automation endpoints, while routes/api.php no longer exposes them. Synchronize the OpenAPI file before publishing it as the production contract.

12 / QUALITY ATTRIBUTES

Security, Non-Functional Requirements, and Deployment

Area	Requirement / current control
Authorization	Default-deny ownership policies; cross-user resources are resolved through owned queries and return 404 where appropriate.
Isolation	Active-project session scoping and same-project relationship validation prevent accidental cross-project display or mutation.
Authentication	Fortify web authentication and Sanctum token authentication; inactive users are rejected by token login.
Input protection	Form Requests and JSON:API validators use allowlisted states, ownership-aware exists rules, MIME/size limits, and relationship checks.
Secrets	Passwords and tokens are hidden/hashed; audit redaction excludes password, token, plain token, and two-factor secret fields.
Availability	Health endpoint /up, queue support, rate limiting, database indexes, and stateless API authentication support scaling.
Performance	Pagination is bounded, analytics use aggregate queries, common filters are indexed, and API includes/sparse fields are allowlisted.
Recoverability	Database backup plus durable export/import; object storage must be backed up independently; users re-login after session recreation.
Maintainability	Controllers delegate to services; policies isolate access decisions; Pest, Larastan, Pint, ESLint, Svelte checks, and Vite build form CI gates.

Deployment topology

Component	Production expectation
Application	Laravel Cloud or equivalent PHP 8.3+ runtime; APP_ENV=production, APP_DEBUG=false, HTTPS APP_URL.
Database	Managed PostgreSQL or MySQL; SSL required for managed PostgreSQL; migrations run before traffic is opened.
Storage	S3-compatible private bucket recommended for issue evidence; local disk is suitable only for local development.
Queue	Long-running or asynchronous work uses a supervised queue worker; failed jobs are monitored.
AI	OpenRouter key and model names are environment configuration; provider failures must not corrupt business records.
API	CORS allowlist, token expiry, rate limits, and API docs visibility are environment-controlled.

Required operational configuration

Core variables include APP_KEY, APP_URL, database credentials/SSL mode, session and cache drivers, filesystem and S3 credentials, ISSUE_ATTACHMENTS_DISK, queue connection, API token/rate/CORS/docs settings, and OpenRouter API key/model settings. Secrets must exist only in environment management, never in source control.

13 / GOVERNANCE

Verification, Risks, and Decision Log

Verification strategy

Test layer	Required coverage
Feature tests	Authentication, ownership isolation, active-project pages, CRUD, relationship constraints, execution lifecycle, issues/files, dashboard/reports, AI normalization, and data migration.
API contract	JSON:API negotiation/errors, token abilities, 404 isolation, pagination/filters/includes/fields, idempotency, ETags, soft deletes, and audit records.
Static and style	Larastan, Pint, ESLint, Prettier, and Svelte type checks.
Build/runtime	Production Vite build, fresh database migration, seed smoke test, storage upload/download, PDF rendering, and health endpoint.
Release smoke	Login -> choose project -> case -> plan -> run -> result -> issue -> report, plus one API token workflow.

Known risks and recommended actions

Priority	Finding	Recommended action
High	OpenAPI contains removed automation paths.	Regenerate or manually reconcile docs/openapi/v1.yaml with routes/api.php before external publication.
High	Two-factor fields are referenced by the User model/security surface but no checked-in migration adds them.	Confirm Fortify feature configuration and add/verify the required schema before enabling 2FA in production.
Medium	Tester, reporter, and assignee are free text.	Keep for single-owner MVP or introduce project membership and user foreign keys as one deliberate migration.
Medium	One issue supports only one image attachment.	Document the constraint in UX; add a one-to-many attachment model only when multiple evidence files are required.
Medium	AI output is provider-dependent.	Retain normalization, user review, clear error handling, timeout controls, and provider observability.
Low	No public restore flow for soft-deleted resources.	Define retention and internal restoration procedure before compliance or support demand grows.

Decision baseline

Decision	Current position
Tenancy	Single owner per project; no memberships or cross-user administration.
Automation	automation_status remains metadata; scripts and browser execution are external to Testify.
API	Additive /api/v1 contract coexists with session endpoints used by the SPA.
Deletion	Primary API resources are soft-deleted and hidden; restoration is internal-only.
Files	Private Laravel disk abstraction, configurable to S3-compatible storage.
AI	Advisory draft generation; user remains responsible for final content and persistence.

Definition of done

Update code, migrations, tests, API documentation, and this specification together whenever a state, relationship, ownership boundary, or external contract changes.